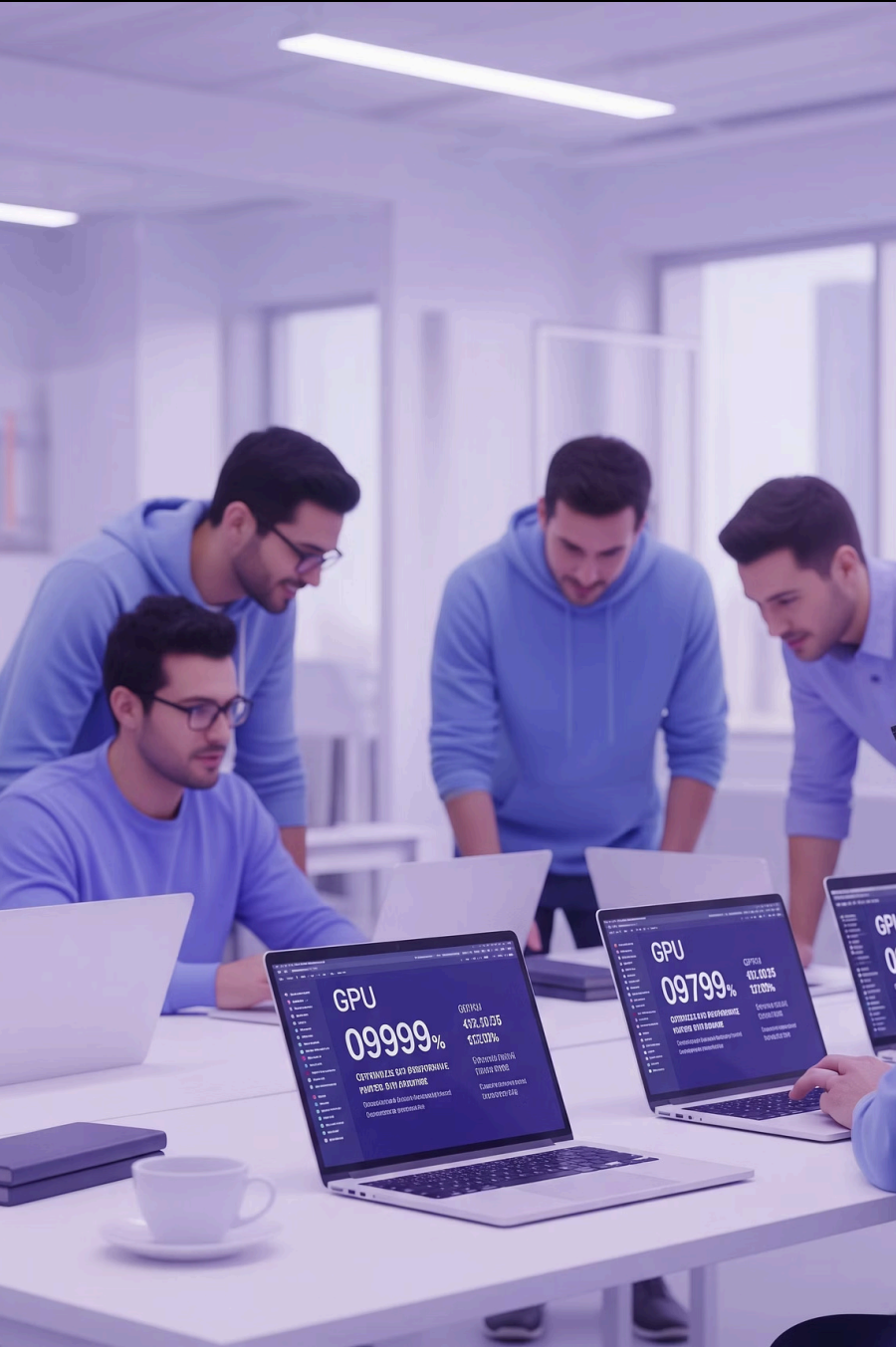




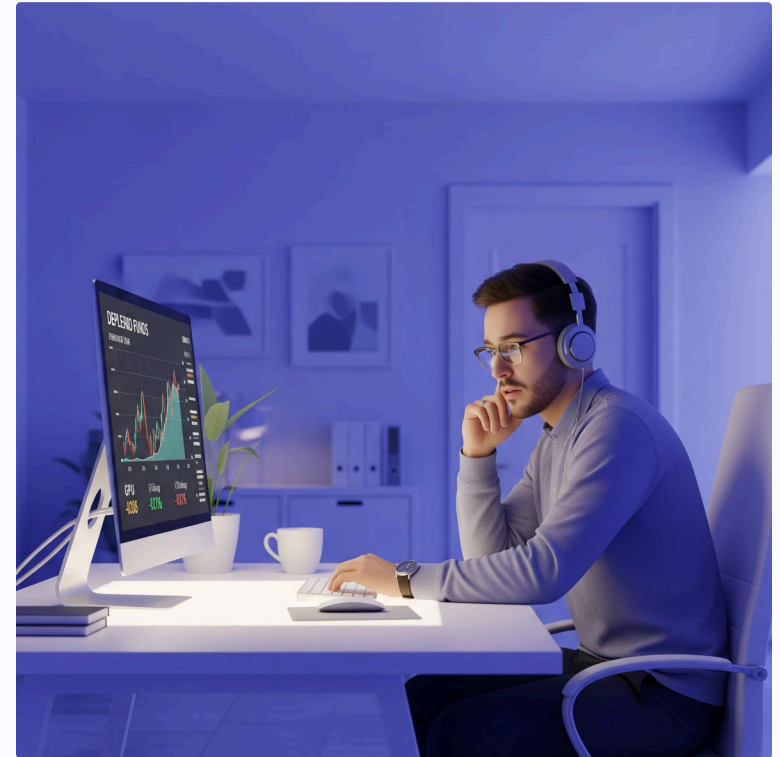
Budget Optimization for Small Teams

Make every GPU dollar count and build a lean infrastructure culture early, effectively balancing **speed vs spend** for maximum runway.

Why Budget Optimization Matters

For startups and small teams, effective AI infrastructure spending can be the difference between success and failure:

- Runway is finite → infrastructure burn rate must be controlled
- Hidden costs quickly accumulate through idle GPUs, egress charges, and duplicate storage
- "More infrastructure ≠ more progress" — smarter usage wins every time



The Big Cost Buckets

Compute

GPU/TPU, CPU, inference serving

Typically the largest expense for ML teams

Storage

Datasets, checkpoints, logs

Can grow unexpectedly with experiment volume

Networking

Data transfer, cross-region operations

Hidden costs that surprise many teams

SaaS/API Usage

LLM APIs, monitoring, CI/CD services

Can scale unpredictably with usage

Compute Cost Optimization

→ Use spot/preemptible GPUs with checkpointing

Save 70-90% on GPU costs by using interruptible instances with proper recovery mechanisms

→ Train small models first → scale only proven ones

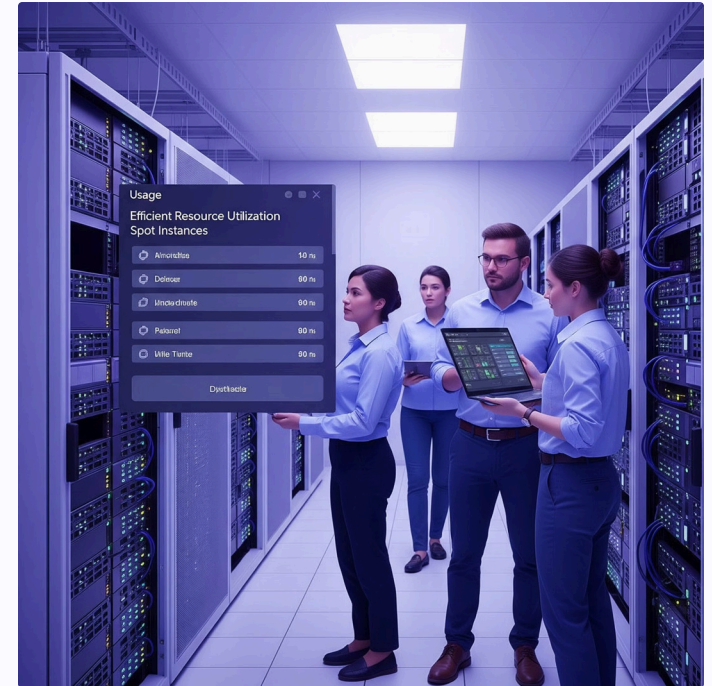
Validate concepts on smaller architectures before scaling to expensive training runs

→ Use mixed precision + quantization

Reduce memory footprint and speed up training by 2-3x while maintaining accuracy

→ Auto-stop idle notebooks and jobs

Implement automatic shutdown of resources after periods of inactivity



Storage Cost Optimization

1

Implement tiered storage strategy

Use SSD for hot data that needs fast access, and cheaper object storage (S3, GCS) for cold data and archives

2

Prune checkpoints & logs automatically

Set up retention policies to automatically delete or archive older checkpoints and logs based on age or performance

3

Deduplicate datasets with DVC

Use Data Version Control (DVC) to manage dataset versions without duplicating large files, saving 30-60% storage

4

Compress models effectively

Convert to optimized formats like ONNX or use quantization to reduce model size by 75% without significant accuracy loss

Networking Cost Optimization

Network costs can be sneaky budget killers for ML workloads:

Co-locate storage + compute in same region

Keep data and processing in the same cloud region to minimize cross-region transfer costs

Minimize cross-region data transfer

Cross-region data movement can cost 10-100x more than in-region; plan architecture accordingly

Use caching layers for repeated dataset access

Implement caching to avoid repeatedly downloading the same data for multiple training runs

Watch out for egress from SaaS APIs

Many cloud services charge for outbound data; monitor API usage patterns closely



People/Process Optimization



Rotate Infra Captain

Assign a rotating role responsible for catching cost leaks and optimizing resource usage each sprint



Cost Dashboards

Implement daily/weekly cost visibility with tools like Kubecost or cloud billing dashboards to create awareness



Tie to Business KPIs

Evaluate experiments based on business metrics, not just model accuracy, to ensure ROI on infrastructure spend



Kill Zombie Jobs

Develop processes to quickly identify and terminate forgotten or non-productive workloads consuming resources

Vendor Strategy



- Start cloud pay-as-you-go

Avoid premature commitments or reserved instances until usage patterns are clear
- Consider specialized GPU providers

RunPod, Lambda, CoreWeave often offer 30-50% lower rates than major clouds for specific workloads
- Negotiate startup credits

AWS, GCP, and Azure all offer startup programs with substantial credits; don't leave money on the table
- Avoid vendor lock-in

Don't become dependent on APIs or services you can't afford at scale

Lean Experimentation Culture



Start Small

Validate ideas on subsets of data before scaling to full datasets



Track Cost Per Experiment

Monitor both model performance and associated infrastructure costs for each run



Use Small Proxies

Test hypotheses on smaller models before committing to expensive large-scale training



Double Down Selectively

Fail cheap and fast, then invest resources only in promising approaches

Creating a culture where efficient experimentation is celebrated builds long-term infrastructure discipline

Cost Guardrails in Practice

Proactive Controls

- Set per-user/team **quotas for GPU usage** to prevent overallocation
- Configure **alerts when spend exceeds thresholds** on daily/weekly basis
- Require jobs exceeding **X hours to checkpoint** or auto-cancel
- Make **budget reviews** a standard part of sprint planning



Effective guardrails prevent surprises while maintaining team velocity

Example: GPU Cost Dashboard

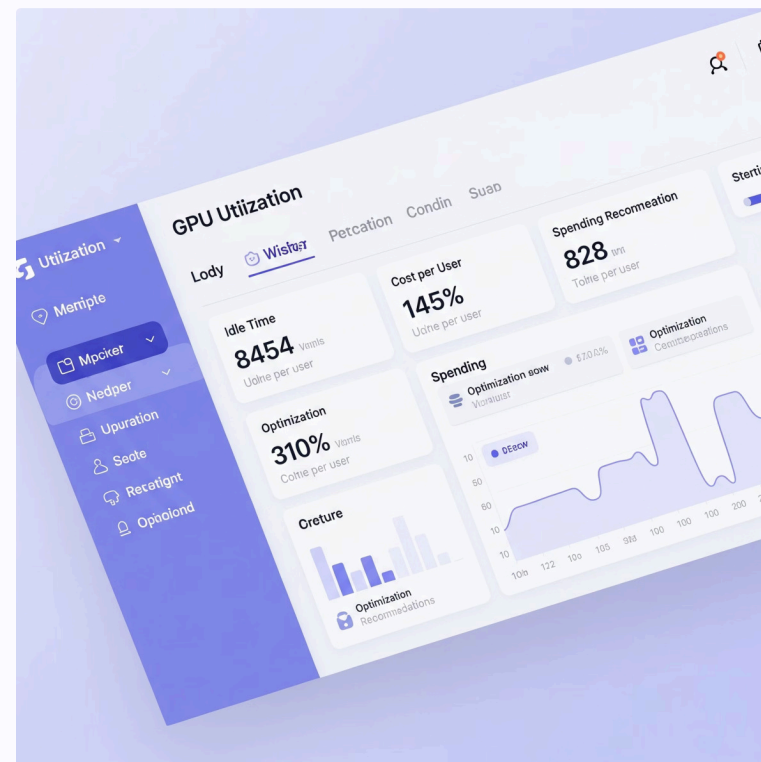
Key Metrics to Track:

- GPU hours consumed per project/team
- Cost attribution per user/experiment
- Idle percentage and utilization rates
- Cost per training run/experiment

Common Insights:

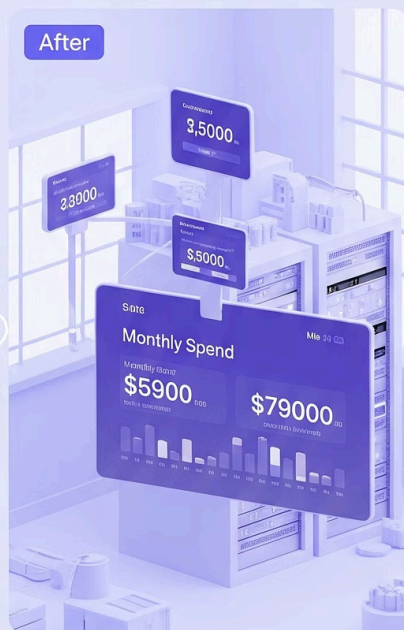
30% of GPU time sitting idle → Opportunity to optimize scheduling and job queueing

Action: Scale down unused nodes, implement auto-shutdown, and migrate suitable workloads to spot instances





Cloud Infrastructure costs



Case Study: Healthcare AI Startup

Initial Situation:

- Early-stage healthcare imaging AI startup
- Burning \$20K/month on on-demand GPUs
- Inconsistent experiment tracking
- Multiple redundant dataset copies

Optimization Strategy:

- Moved 80% of workloads to spot instances with checkpointing
- Implemented MLflow + Prefect (open-source stack)
- Added idle detection and automatic shutdown
- Result: **\$7K monthly savings**
= 6 more months of runway

Budget Optimization Framework

Compute

Use spot instances and pruning

Networking

Optimize egress and caching

Storage

Apply caching and tiering

SaaS

Leverage credits and rightsizing



Strategic cost management isn't just about saving money—it's about **allocating resources where they drive the most value** for your ML initiatives.

Summary: Make Every Dollar Count

Priority Cost Areas

Budget drains: compute > storage
> networking > SaaS

Focus optimization efforts where
spending is highest first

Key Strategies

Optimize with **spot GPUs,**
pruning, co-location, OSS stack

Build cost awareness into your
team's culture from day one

Business Impact

Savings = longer runway + more
experiments

Disciplined ML infra spending can
literally save your startup

Questions? Connect with us for personalized infrastructure optimization guidance